

Algoritmi e Strutture Dati

Algoritmi probabilistici

Alberto Montresor

Università di Trento

2026/08/15

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

Audentes fortuna iuvat – Virgilio

Quando non sapete quale strada prendere, una scelta casuale può essere una strategia

- 1 Introduzione
- 2 Algoritmi Monte Carlo
 - Espressione polinomiale nulla
 - Test di primalità
 - Strutture dati probabilistiche
- 3 Algoritmi Las Vegas
 - Problema della selezione

Fino a questo punto del corso

Casualità nel modello di input

L'algoritmo è deterministico, ma viene analizzato assumendo una distribuzione di probabilità sugli input

- Si calcola una media sui possibili dati di ingresso, pesata secondo tale distribuzione
- Esempio: analisi del caso medio di Quicksort
- Tipicamente: tutte le permutazioni sono equiprobabili

Algoritmi probabilistici

Scelte casuali dell'algoritmo

L'algoritmo usa numeri casuali durante l'esecuzione

- La stessa istanza può produrre esecuzioni diverse
- La probabilità influenza l'output prodotto o il tempo di esecuzione
- Esempio: scelta casuale di un valore di test

Due tipologie di algoritmi

- negli algoritmi **Monte Carlo**, la correttezza dell'output è probabilistica
- negli algoritmi **Las Vegas**, l'output è sempre corretto, ma il tempo di esecuzione è probabilistico

Espressione polinomiale nulla

Problema

Data un'espressione algebrica polinomiale $p(x_1, \dots, x_n)$ in n variabili, determinare se p è **identicamente nulla**

Esempi

$$p(x, y) = (x + y)^2 - x^2 - 2xy - y^2 \equiv 0 \quad \text{Identicamente nulla}$$

$$q(x, y) = (x + y)^2 - x^2 - 2xy = y^2 \not\equiv 0 \quad \text{Non nulla}$$

Discussione

- Se il polinomio fosse già in forma di monomi, il problema sarebbe semplice
- In generale, espandere e semplificare simbolicamente può essere costoso

Espressione polinomiale nulla

Algoritmo

- Si genera una n -pla di valori $v_1, \dots, v_n$
- Si calcola $x = p(v_1, \dots, v_n)$
 - Se $x \neq 0$, allora p **non** è identicamente nulla, **return false**
 - Se $x = 0$, allora p **potrebbe** essere identicamente nulla
- Per il Lemma di Schwartz–Zippel, se $p \neq 0$, ha **grado totale**^a d , e ogni v_i è scelto uniformemente e indipendentemente in $\{1, \dots, 2d\}$, allora

$$\Pr[p(v_1, \dots, v_n) = 0] \leq \frac{d}{2d} = \frac{1}{2}$$

^aIl **grado totale di un monomio** è la somma degli esponenti delle sue variabili; il **grado totale di un polinomio** è il massimo grado totale dei suoi monomi.

Espressione polinomiale nulla

Ripetizione del test

- Ripetiamo il test k volte con scelte indipendenti
- Se almeno una valutazione è diversa da zero, restituiamo **false**
- Se tutte le valutazioni sono uguali a zero, restituiamo **true**

Esito dei test	Output	Garanzia
$\exists j : p(v_1^{(j)}, \dots, v_n^{(j)}) \neq 0$	false	sicuramente corretto
$\forall j : p(v_1^{(j)}, \dots, v_n^{(j)}) = 0$	true	errore possibile

Errore

Se $p \neq 0$, la probabilità di restituire erroneamente **true** è al più $\left(\frac{1}{2}\right)^k$

Espressione polinomiale nulla

Questa tecnica permette di sostituire una verifica simbolica costosa con una verifica probabilistica basata su valutazioni casuali.

L'errore è **unilaterale**: se l'algoritmo restituisce **false**, la risposta è certamente corretta; se restituisce **true**, la risposta è corretta con alta probabilità.

Appare, ad esempio, nei test di prodotti di matrici e in algoritmi randomizzati per problemi su grafi.

Test di primalità (Primality test)

Test di primalità

Determinare se un numero in input n è primo

```
boolean isPrimeNaif(int n)
```

```
if  $n < 2$  then
```

```
    return false
```

```
else
```

```
    for  $i = 2$  to  $\lfloor \sqrt{n} \rfloor$  do
```

```
        if  $n \bmod i == 0$  then
```

```
            return false
```

```
    return true
```

- Testa tutti gli interi fra 2 e $\lfloor \sqrt{n} \rfloor$
- Costo: $\mathcal{O}(\sqrt{n})$ test di divisibilità
- Se $\ell = \lceil \log_2(n+1) \rceil$ è la dimensione dell'input, allora $\mathcal{O}(\sqrt{n}) = \mathcal{O}(2^{\ell/2})$
- Quindi il metodo non è polinomiale nella dimensione dell'input

Test di primalità – Fermat

Piccolo teorema di Fermat

Se n è primo, allora:

$$\forall b, 2 \leq b < n : \quad b^{n-1} \bmod n = 1$$

Test di primalità di Fermat

```
boolean isPrime1(int n)
```

```
  b = random(2, n - 1)
```

```
  if  $b^{n-1} \bmod n \neq 1$  then
```

```
    return false
```

```
  return true
```

Questo algoritmo è corretto?

Esistono numeri composti (**pseudoprimi** in base b) tali che:

$$\exists b, 2 \leq b < n : b^{n-1} \bmod n = 1$$

Esempio :

$$n = 341 = 11 \times 13$$

$$2^{340} \bmod 341 = 1$$

Test di primalità – Fermat

Piccolo teorema di Fermat

Se n è primo, allora:

$$\forall b, 2 \leq b < n : \quad b^{n-1} \bmod n = 1$$

Test di primalità di Fermat

Questo algoritmo è corretto?

```
boolean isPrime2(int n)
```

```
for  $i = 1$  to  $k$  do
```

```
     $b = \text{random}(2, n - 1)$ 
```

```
    if  $b^{n-1} \bmod n \neq 1$  then
```

```
        return false
```

```
return true
```

Output	n
false	sicuramente composto
true	possibilmente primo

Test di primalità – Fermat

Piccolo teorema di Fermat

Se n è primo, allora:

$$\forall b, 2 \leq b < n : \quad b^{n-1} \bmod n = 1$$

Test di primalità – Miller-Rabin

Teorema

Sia $n > 2$ un intero dispari e sia $n - 1 = m \cdot 2^v$ con m dispari. Se n è **primo**, allora **per ogni** intero b , con $2 \leq b < n$, valgono **entrambe** le seguenti condizioni:

- 1 $\text{mcd}(n, b) = 1$
- 2 $b^m \bmod n = 1 \vee \exists i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n = n - 1$

Contrapposizione

Se **esiste** un intero b , con $2 \leq b < n$ tale che **almeno una** delle seguenti condizioni è vera:

- 1 $\text{mcd}(n, b) \neq 1$
- 2 $b^m \bmod n \neq 1 \wedge \forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$

allora n è **composto**

Test di primalità – Miller-Rabin

Verifica primalità n – Esempio: $n = 221 = 13 \times 17$

Sia n un numero dispari diverso da 1

$$n = 221$$

Sia $n - 1 = m \cdot 2^v$, con m dispari

$$220 = 55 \cdot 2^2$$

$n - 1$ in binario è pari a:

m in binario seguito da v zeri

$$\overbrace{110111}^{m=55} \underbrace{00}_{v=2}$$

se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è **composto**:

$$b = 137 \text{ (Witness)}$$

$$(1) \gcd(n, b) \neq 1$$

$$\gcd(221, 137) = 1$$

$$(2) b^m \bmod n \neq 1 \text{ and}$$

$$137^{55} \bmod 221 = 188 \neq 1$$

$$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$$

$$137^{55 \cdot 2^0} \bmod 221 = 188 \neq 220$$

$$137^{55 \cdot 2^1} \bmod 221 = 205 \neq 220$$

Test di primalità – Miller-Rabin

Verifica primalità n – Esempio: $n = 229$ numero primo

Sia n un numero dispari

$$n = 229$$

Sia $n - 1 = m \cdot 2^v$, con m dispari

$$228 = 57 \cdot 2^2$$

$n - 1$ in binario è pari a:

m in binario seguito da v zeri

$$\overbrace{111001}^{m=57} \underbrace{00}_{v=2}$$

se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è **composto**:

Proviamo $b = 137$

(1) $\text{mcd}(n, b) \neq 1$

$$\text{mcd}(229, 137) = 1$$

(2) $b^m \bmod n \neq 1$ **and**

$$137^{57} \bmod 229 = 122 \neq 1$$

$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$

$$137^{57 \cdot 2^0} \bmod 229 = 122 \neq 228$$

$$137^{57 \cdot 2^1} \bmod 229 = \textcolor{red}{228}$$

Test di primalità – Miller-Rabin

Verifica primalità n – Esempio: $n = 221 = 13 \times 17$

Sia n un numero dispari

$$n = 221$$

Sia $n - 1 = m \cdot 2^v$, con m dispari

$$220 = 55 \cdot 2^2$$

$n - 1$ in binario è pari a:

m in binario seguito da v zeri

$$\overbrace{110111}^{m=55} \underbrace{00}_{v=2}$$

se $\exists b, 2 \leq b < n$ per cui una delle seguenti affermazioni è vera, n è **composto**:

$$b = 174 \text{ (Strong liar)}$$

$$(1) \text{ mcd}(n, b) \neq 1$$

$$\text{mcd}(221, 174) = 1$$

$$(2) b^m \bmod n \neq 1 \text{ and}$$

$$174^{55} \bmod 221 = 47 \neq 1$$

$$\forall i, 0 \leq i < v : b^{m \cdot 2^i} \bmod n \neq n - 1$$

$$174^{55 \cdot 2^0} \bmod 221 = 47 \neq 220$$

$$174^{55 \cdot 2^1} \bmod 221 = \textcolor{red}{220}$$

Test di primalità – Miller-Rabin

- Per ogni n dispari composto, al più un quarto delle basi $b \in \{2, \dots, n-2\}$ non rivela la compostezza di n .
- Quindi una singola iterazione del test ha probabilità di errore al più $1/4$.
- Ha senso per valori n dispari di grandi dimensioni

```
boolean isPrime(int  $n$ )
```

```
for  $i = 1$  to  $k$  do
```

```
     $b = \text{random}(2, n-2)$ 
```

```
    % isComposite() restituisce true se  $b$  è un witness per  $n$ 
```

```
    if isComposite( $n, b$ ) then
```

```
        return false
```

```
return true
```

Test di Miller-Rabin

Riassunto

- Complessità: $\mathcal{O}(k \log^2 n \log \log n \log \log \log n)$
- Probabilità di errore: $(1/4)^k$
- Algoritmo di tipo Monte Carlo

Algoritmi probabilistici vs algoritmi deterministici

- Dal 2002, esiste l'algoritmo deterministico AKS di complessità $\mathcal{O}(\log^{6+\epsilon} n)$
- I fattori moltiplicativi coinvolti sono molto alti
- Si preferisce quindi l'algoritmo di Miller-Rabin

BitSet + Tabelle Hash = Bloom Filters

BitSet

Vantaggi: 1 bit per ogni possibile oggetto dell'universo

Svantaggi: Elenco prefissato di oggetti

Tabelle Hash

Vantaggi: Struttura dati dinamica

Svantaggi: Alta occupazione di memoria

Bloom filters

Vantaggi

- Supporta inserimenti (non cancellazioni)
- Bassa occupazione di memoria (10 bit/oggetto)

Svantaggi

- Niente cancellazioni
- Risposta probabilistica: possibili falsi positivi
- Non permette di recuperare gli elementi inseriti

Bloom filter

Specifica

Un Bloom filter rappresenta in modo compatto un insieme S di chiavi inserite.

- **insert**(key): inserisce key nell'insieme S
- **boolean contains**(key): verifica se $key \in S$
 - Se restituisce **false**, allora $key \notin S$ sicuramente
 - Se restituisce **true**, allora $key \in S$ forse: possono verificarsi **falsi positivi**

Bloom filter

Trade-off fra occupazione di memoria e probabilità di falso positivo

- Sia ϵ la probabilità di falso positivo
- I bloom filter richiedono $1.44 \log_2(1/\epsilon)$ bit per elemento inserito

ϵ	Bit
10^{-1}	4.78
10^{-2}	9.57
10^{-3}	14.35
10^{-4}	19.13

Applicazioni dei Bloom Filter

Pattern comune

- Si mantiene localmente una rappresentazione compatta di un insieme S
- Se $\text{contains}(x) = \text{false}$, allora $x \notin S$ sicuramente
- Se $\text{contains}(x) = \text{true}$, si esegue una verifica più costosa

Applicazione	Operazione evitata
Database / storage engine	Lettura da disco
Sicurezza web	Verifica di rete
Blockchain	Scansione di blocchi

Applicazioni dei Bloom Filter

Storage engine e database

- **Apache Cassandra**: evita di controllare SSTable che sicuramente non contengono la partizione cercata
- **RocksDB**: associa Bloom filter agli SST file per accelerare le ricerche puntuali

Sicurezza web

- **CRLite**: usa Bloom filter in cascata per comprimere informazioni di revoca dei certificati
- Obiettivo: ridurre verifiche di rete preservando efficienza e privacy

Blockchain

- **Ethereum**: Bloom filter nei log per individuare rapidamente blocchi o transazioni potenzialmente rilevanti

Implementazione

- Un vettore booleano A di m bit, inizializzato a **false**
- k funzioni hash $h_1, h_2, \dots, h_k : U \rightarrow [0, m - 1]$

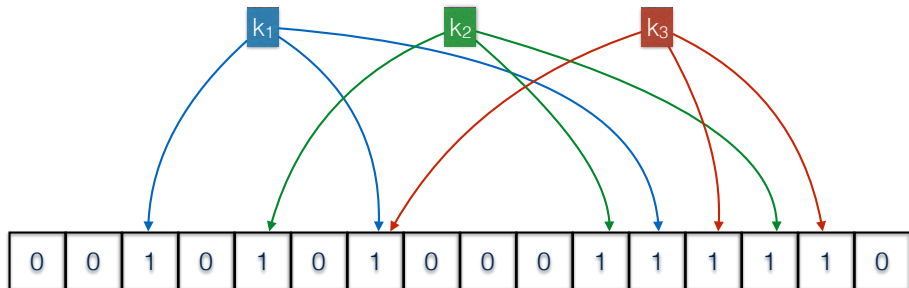
 k_1 k_2 k_3 

Implementazione

```
insert(key)
```

```
for  $i = 1$  to  $k$  do
```

```
   $A[h_i(\textit{key})] = \text{true}$ 
```



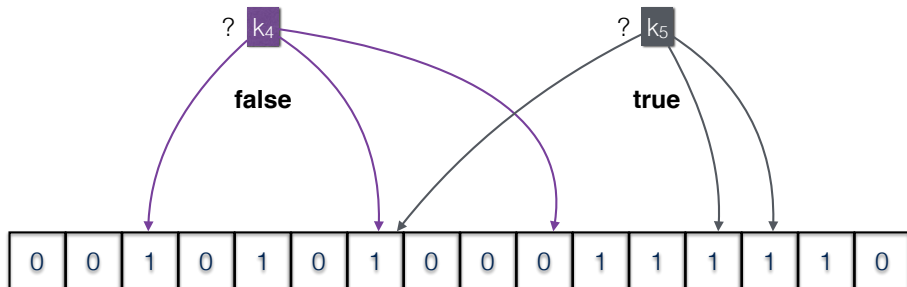
Implementazione

```
boolean contains(key)
```

```
for  $i = 1$  to  $k$  do
```

```
    if  $A[h_i(\textit{key})] == \textit{false}$  then return false
```

```
return true
```



Qualche formula (senza dimostrazione)

Le formule seguenti valgono nel modello ideale in cui le funzioni hash producono posizioni uniformi e indipendenti in $[0, m - 1]$

- Dati n oggetti, m bit, k funzioni hash, la probabilità di un falso positivo è pari a:

$$\epsilon = \left(1 - e^{-kn/m}\right)^k$$

- Dati n oggetti e m bit, il valore ottimale per k è pari a

$$k = \frac{m}{n} \ln 2$$

- Dati n oggetti e una probabilità di falsi positivi ϵ , il numero di bit m richiesti è pari a:

$$m = -\frac{n \ln \epsilon}{(\ln 2)^2}$$

Statistica

Algoritmi statistici su vettori

Estraggono alcune caratteristiche statisticamente rilevanti da un vettore numerico

Esempi

- **Media:** $\mu = \frac{1}{n} \sum_{i=0}^{n-1} A[i]$
- **Varianza:** $\sigma^2 = \frac{1}{n} \sum_{i=0}^{n-1} (A[i] - \mu)^2$
- **Moda:** il valore (o i valori) più frequenti

Statistiche d'ordine

Selezione

Dato un array A contenente n valori e un valore $1 \leq k \leq n$, trovare l'elemento che occuperebbe la k -esima posizione* se il vettore fosse ordinato

Mediana

Il problema del calcolo della mediana è un sottoproblema del problema della selezione con $k = \lceil n/2 \rceil$.

Ovviamente, possiamo ordinare i valori e andare a cercare il valore in posizione k , con costo $\mathcal{O}(n \log n)$. Possiamo fare meglio di così?

* Useremo k come **rango 1-based**: $k = 1$ indica il minimo, $k = n$ indica il massimo

Selezione per piccoli valori di k

Intuizione

- Si può utilizzare uno heap
- L'algoritmo può essere generalizzato a valori generici di $k > 2$

```
int heapSelect(int []  $A$ , int  $n$ , int  $k$ )
```

```
  buildHeap( $A$ )
```

```
  for  $i = 1$  to  $k - 1$  do
```

```
    | deleteMin( $A$ ,  $n$ )
```

```
    |  $n = n - 1$ 
```

```
  return deleteMin( $A$ ,  $n$ )
```

Complessità

- $\mathcal{O}(n + k \log n)$
- Se $k = \mathcal{O}(n / \log n)$,
il costo è $\mathcal{O}(n)$
- Se $k = n/2$, non
va bene

Idea

- Approccio divide-et-impera simile al Quicksort
- Non dovendo ordinare tutto il vettore, dopo la partizione è sufficiente ricorrere in una sola delle due parti
- La scelta della parte dipende dal rango k dell'elemento cercato
- Bisogna fare attenzione agli indici

Algoritmo di selezione

```
int selection(int[] A, int start, int end, int k)
```

```
  if start == end then
```

```
    return A[start]
```

```
  else
```

```
    int j = pivot(A, start, end)
```

```
    int q = j - start + 1
```

```
    if k == q then
```

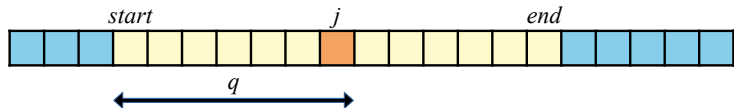
```
      return A[j]
```

```
    else if k < q then
```

```
      return selection(A, start, j - 1, k)
```

```
    else
```

```
      return selection(A, j + 1, end, k - q)
```



Complessità

Caso pessimo

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n-1) + n & n > 1 \end{cases}$$
$$T(n) = \mathcal{O}(n^2)$$

Caso ottimo

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(n/2) + n & n > 1 \end{cases}$$
$$T(n) = \mathcal{O}(n)$$

Complessità

Tempo atteso

Assumiamo che `pivot()` restituisca con la stessa probabilità una qualsiasi posizione j del vettore A

$$T(n) = n + \frac{1}{n} \sum_{q=1}^n T(\max\{q-1, n-q\}) \quad \text{Media su } n \text{ casi}$$

$$\leq n + \frac{1}{n} \sum_{q=\lfloor n/2 \rfloor}^{n-1} 2T(q), \quad \text{per } n > 1$$

Esempi

- $n = 4$: $\max\{0, 3\} + \max\{1, 2\} + \max\{2, 1\} + \max\{3, 0\}$
- $n = 5$: $\max\{0, 4\} + \max\{1, 3\} + \max\{2, 2\} + \max\{3, 1\} + \max\{4, 0\}$

Complessità

$$T(n) \leq n + \frac{1}{n} \sum_{q=\lfloor \frac{n}{2} \rfloor}^{n-1} 2 \cdot cq \leq n + \frac{2c}{n} \sum_{q=\lfloor \frac{n}{2} \rfloor}^{n-1} q$$

Sostituzione, raccolgo $2c$

$$= n + \frac{2c}{n} \left(\sum_{q=1}^{n-1} q - \sum_{q=1}^{\lfloor \frac{n}{2} \rfloor - 1} q \right)$$

Sottrazione prima parte

$$= n + \frac{2c}{n} \cdot \left(\frac{n(n-1)}{2} - \frac{(\lfloor \frac{n}{2} \rfloor)(\lfloor \frac{n}{2} \rfloor - 1)}{2} \right)$$

$$\leq n + \frac{2c}{n} \cdot \left(\frac{n(n-1)}{2} - \frac{(\frac{n}{2} - 1)(\frac{n}{2} - 2)}{2} \right)$$

Rimozione limite inferiore

$$= n + \frac{2c}{n} \cdot \frac{(n^2 - n - (\frac{1}{4}n^2 - \frac{3}{2}n + 2))}{2}$$

$$= n + \frac{c}{n} \cdot \left(\frac{3}{4}n^2 + \frac{1}{2}n - 2 \right)$$

$$\leq n + \frac{c}{n} \cdot \left(\frac{3}{4}n^2 + \frac{1}{2}n \right) = n + \frac{3}{4}cn + \frac{1}{2}c \stackrel{?}{\leq} cn \quad \text{Vera per } c \geq 6 \text{ e } n \geq 6$$

Complessità

- Siamo partiti dall'assunzione
 - j assume equiprobabilisticamente tutti i valori compresi fra $start$ e end
- E se non fosse vero?
- Lo forziamo noi!
 - $A[\text{random}(start, end)] \leftrightarrow A[start]$
- Questo accorgimento vale anche per QuickSort
- La complessità nel caso medio:
 - $\mathcal{O}(n)$ nel caso della Selezione
 - $\mathcal{O}(n \log n)$ nel caso dell'Ordinamento

Selezione deterministica

Algoritmo black-box

Supponiamo di avere un algoritmo “black box” che restituisca il mediano di n valori in tempo $\mathcal{O}(n)$

Domande

- Potrei utilizzarlo per ottimizzare il problema della selezione?
- Che complessità otterrei?

Selezione deterministica

Se conoscessi tale algoritmo

- il problema della selezione sarebbe quindi risolto...
- ... ma dove lo trovo un simile algoritmo?

Rilassiamo le nostre pretese

- Supponiamo di avere un algoritmo “black box” che restituisca un valore x il cui rango è compreso tra $\frac{3}{10}n$ e $\frac{7}{10}n$.
- Potrei utilizzarlo per ottimizzare il problema della selezione?
- Che complessità otterrei?

Blum, Floyd, Pratt, Rivest e Tarjan (1973)

Idea

- Suddividi i valori in gruppi di 5. Chiameremo l' i -esimo gruppo S_i , con $i \in [1, \lceil n/5 \rceil]$
- Trova il mediano M_i di ogni gruppo S_i
- Tramite una chiamata ricorsiva, trova il mediano m delle mediane $[M_1, M_2, \dots, M_{\lceil n/5 \rceil}]$
- Usa m come pivot e richiama l'algoritmo ricorsivamente sull'array opportuno, come nella [selection\(\)](#) randomizzata
- Quando la dimensione scende sotto una certa soglia, possiamo utilizzare un algoritmo di ordinamento per trovare il mediano

Blum, Floyd, Pratt, Rivest e Tarjan (1973)

```
int select(int[] A, int start, int end, int k)
```

```
int n = end - start + 1
```

```
% Se la dimensione è inferiore ad una soglia (10), ordina il vettore e  
% restituisci il  $k$ -esimo elemento di  $A[start \dots end]$ 
```

```
if n  $\leq$  10 then
```

```
    InsertionSort(A, start, end)           % Versione con indici inizio/fine  
    return A[start + k - 1]
```

```
% Divide A in  $\lceil n/5 \rceil$  sottovettori di dim. 5 e ne calcola la mediana
```

```
M = new int[0 ...  $\lceil n/5 \rceil - 1$ ]
```

```
for i = 0 to  $\lceil n/5 \rceil - 1$  do
```

```
    M[i] = median5(A, start + i * 5, min(start + i * 5 + 4, end))
```

```
% Individua la mediana delle mediane e usala come perno
```

```
int m = select(M, 0,  $\lceil n/5 \rceil - 1$ ,  $\lceil \lceil n/5 \rceil / 2 \rceil$ )
```

```
int j = pivot(A, start, end, m)           % Versione con m in input
```

```
[...]
```

Blum, Floyd, Pratt, Rivest e Tarjan (1973)

```
int select(int[] A, int start, int end, int k)
```

```
[...]
```

% j è l'indice del pivot; q è il suo rango relativo nel sottovettore

% Confronta q con il rango cercato e restituisce il valore conseguente

```
int  $q = j - start + 1$ 
```

```
if  $k == q$  then
```

```
    | return  $A[j]$ 
```

```
else if  $k < q$  then
```

```
    | return select( $A, start, j - 1, k$ )
```

```
else
```

```
    | return select( $A, j + 1, end, k - q$ )
```

Blum, Floyd, Pratt, Rivest e Tarjan (1973)

- Il calcolo dei mediani $M[]$ richiede al più $6\lceil n/5 \rceil$ confronti.
- La prima chiamata ricorsiva dell'algoritmo `select()` viene effettuata su $\lceil n/5 \rceil$ elementi
- La seconda chiamata ricorsiva dell'algoritmo `select()` viene effettuata al più su $\frac{7}{10}n$ elementi
- Ignorando gli arrotondamenti, l'algoritmo `select()` esegue nel caso pessimo $\mathcal{O}(n)$ confronti:

$$T(n) \leq T(n/5) + T(7n/10) + \frac{11}{5}n$$

1	4	7	10	13	2	5	8	11	14	3	6	9	12	15
---	---	---	----	----	---	---	---	----	----	---	---	---	----	----

Conclusioni

Quale preferire?

- Algoritmo probabilistico Las Vegas in tempo atteso $\mathcal{O}(n)$
- Algoritmo deterministico in tempo $\mathcal{O}(n)$, con fattori moltiplicativi più alti